

Implementations of ML algorithms and explanation with SHAP

Group 3

Indian Institute of information technology, Sri City

Roll Numbers: S20230010225, S20230010224, S2023001032, S20230010110

Names: Shriansh Jain, Shreyaan Loke, Ayush, Siva Sampreeth

email: {shriansh.j23, shreyaan.l23, ayush.p23, sivasampreeth.j23}@iiits.in

Abstract—Our Term Report is on the project where we have implemented different machine learning algorithms from scratch, and then tried to add SHAP (SHapley Additive exPlanations) to them. The algorithms we have implemented are K-Nearest Neighbors (KNN), Logistic Regression, Artificial Neural Networks (ANN), and Isolation Forest algorithms. Each implementation is tested out with different datasets: UCI Spambase for KNN, Pima Indians Diabetes for Logistic Regression, Telco Customer Churn for ANN, and UCI Air Quality for Isolation Forest. The usage of SHAP provides us with a little more interpretable explanations of a models prediction, and is very useful in the case of some more black box like models.

I. INTRODUCTION

Machine Learning is obviously a very important part of a lot of applications these days, and we can see it being used in various industries including but not limited to spam detection, medical diagnosis and fraud detection.

However, there is a major gap, the black box nature of a lot of models poses challenges for trust, debugging, and compliance with some regulations. Our Project addresses this challenge by adopting SHAP (SHapley Additive exPlanations) to bridge it.

A. Motivation

We have identified 2 major reasons for having chosen this project:

- while it is a project requirement, implementing these algorithms was beneficial us to understand the actual working rather than just theory.
- the need for explainable AI has grown significantly, particularly in sensitive domains like healthcare and finance where stakeholders require justification for automated decisions.

B. Contributions

Our key contributions include:

- From-scratch implementations of four ML algorithms without using Inbuilt functions
- Usage of these algorithms on real datasets that we pre-processed ourselves
- Integration of SHAP for model-agnostic explanations
- Comparative analysis revealing algorithm strengths for specific data types

II. RELATED WORK

A. Algorithms

This work builds upon foundational machine learning models known for their interpretability, scalability, and relevance across domains. K-Nearest Neighbors (KNN), introduced by Fix and Hodges (1951), remains a simple yet powerful non-parametric method, relying on distance metrics to classify based on local neighborhoods. Logistic regression, dating back to the 1950s, provides a probabilistic framework for binary classification, with interpretable feature weights and well-understood decision boundaries.

Artificial Neural Networks (ANNs), originating from the perceptron model (Rosenblatt, 1958), have evolved into multi-layer architectures capable of capturing non-linear relationships. The introduction of backpropagation enabled efficient training of deep networks, making ANNs a versatile tool for supervised learning.

Isolation Forests, proposed here [2], offer a tree-based approach to anomaly detection by explicitly isolating outliers through recursive partitioning. Unlike density-based methods, they scale well to high-dimensional data and require fewer assumptions about the underlying distribution.

B. Anomaly Detection

Isolation Forest, introduced by Liu et al. (2008) [2], revolutionized anomaly detection by exploiting the principle that anomalies are "few and different," requiring fewer random splits to isolate. This approach outperforms density-based methods on high-dimensional data.

C. Explainable AI

SHAP [1], based on Shapley values from game theory, provides unified model agnostic explanations. SHAP offers consistency and local accuracy guarantees. The SHAP library also extends SHAP to various model types, including tree-based and neural architectures. While very slow for some, the algorithm is able to give explanations for any model.

III. DATASETS AND PREPROCESSING

We evaluate our implementations on four different datasets, each representing a ML task.

A. UCI Spambase Dataset

The Spambase dataset contains 4,601 email samples with 57 features representing different things like word frequencies, character frequencies, and capital letter metrics. The binary target indicates spam (39%) or legitimate email (61%).

Preprocessing involved:

- Missing value imputation using mean
- Min max scaling to normalize feature ranges
- Train-test split (80-20)

B. Pima Indians Diabetes Dataset

This dataset comprises 768 patient records with 8 medical features (glucose, BMI, age, etc.) predicting diabetes onset. Class distribution: 268 positive (34.9%), 500 negative (65.1%).

Preprocessing involved:

- Replaced zero values in biological features with NaN (eg: You cant have skin thickness of 0)
- Imputed missing values using mean(Glucose, BloodPressure) and median(SkinThickness, Insulin, BMI).
- Features scaled using StandardScaler

C. Telco Customer Churn Dataset

The Telco dataset contains 7,043 customer records with 19 features covering demographics, services, and billing information. Target: churned (27%) vs. no churn (73%).

Preprocessing involved:

- Dropping ID column
- One-hot encoding for categorical variables
- Label encoding for binary features
- Converted TotalCharges to numeric and impute with median
- Scaling using standard scaler

D. UCI Air Quality Dataset

Time-series data with 9,471 hourly measurements of air pollutants (CO, NOx, NO2, etc.) and meteorological variables(17 columns).

For anomaly detection, we had to do significantly more preprocessing:

- Handled missing values (-200.0) replacement with NaN
- NMHC column dropped as majority values were Null
- Visualised the time series data to see the trends throughout a week/month/day
- Scaled the dataframe using standard scaler(Note that we did not normalize, or that would remove some anomalies itself)
- Created cyclical features (sin/cos) for temporal columns()
- Dropped original time columns

IV. ALGORITHM IMPLEMENTATIONS

A. K Nearest Neighbors Classifier

Our KNN implementation uses Euclidean distance, and majority class is chosen:

Algorithm: For test point $\mathbf{x}$:

- 1) Compute euclidean distance: $d_i = \|\mathbf{x} - \mathbf{x}_i\|^2$ for all training points
- 2) Select k nearest neighbors (k is hyperparameter)
- 3) Return majority class among these k neighbors

B. Logistic Regression

Sigmoid Activation Function: The sigmoid function maps the linear combination of input features to a probability value between 0 and 1:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

where $z = Xw + b$, X represents the input features, w the weights, and b is the bias.

Training Process: The model parameters (w and b) are optimized using gradient descent. The loss function chosen is binary cross entropy:

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

where $\hat{y}_i$ is the predicted probability, y_i is the true label, and n is the number of samples.

The gradients of the loss function are computed as:

$$\frac{\partial L}{\partial w} = \frac{1}{n} X^T (\hat{y} - y), \quad \frac{\partial L}{\partial b} = \frac{1}{n} \sum (\hat{y} - y)$$

The weights and bias are updated iteratively:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

where η is the learning rate.

Visualization: The training loss curve demonstrated the model's convergence over iterations, and the confusion matrix provided a clear representation of the classification results.

C. Artificial Neural Network Implementation

Artificial Neural Networks (ANNs) are models capable of learning complex patterns in data. We have built our own ANN from scratch but still made use of Pytorch tensors for efficiency.

Model Architecture: The ANN consists of:

- **Input Layer:** Accepts the processed features of the dataset.
- **Hidden Layers:** Two layers with 14 and 8 neurons, respectively, using the ReLU activation function.
- **Output Layer:** A single neuron with a sigmoid activation function to predict the probability of churn.

Weight Initialization: Weights are initialized using the He initialization method to improve convergence:

$$w \sim \mathcal{N}(0, \sqrt{\frac{2}{\text{fan_in}}})$$

Forward Propagation: The input data is passed through the network, with the ReLU activation applied in the hidden layers and the sigmoid activation in the output layer.

Loss Function: The Binary Cross-Entropy Loss is used to measure the difference between predicted probabilities and true labels:

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Training Process: The model is trained using Stochastic Gradient Descent. Gradients are computed and weights are updated iteratively:

$$w = w - \eta \frac{\partial L}{\partial w}, \quad b = b - \eta \frac{\partial L}{\partial b}$$

where η is the learning rate.

D. Isolation Forest

We have implemented a Isolation Forest for anomaly detection in air quality time series data.

The custom Isolation Forest implementation follows the original algorithm proposed here:

- 1) **Tree Structure:** We use a binary tree structure with nodes of 2 types, `LeafNode` and `DecisionNode`, where decision node contains split attributes and values, while leaf nodes store sample size and data information.
- 2) **Height Limitation:** Trees are constrained by $height_limit = \lceil \log_2(sample_size) \rceil$ to prevent overfitting and ensure efficiency.
- 3) **Anomaly Score Calculation:** The anomaly score for each instance is computed using:

$$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}} \quad (1)$$

where $E(h(x))$ is the average path length of instance x over all trees, and $c(n)$ represents the average path length of unsuccessful search in a BST with n points.

- 4) **Model Configuration:** The algorithm was configured with the following parameters for the air quality dataset:
 - **Number of trees:** 100
 - **Sample size per tree:** $\min(256, n_{samples})$
 - **Contamination rate:** 1%

V. EXPERIMENTAL RESULTS

A. K-Nearest Neighbors on Spambase

Below Table summarizes KNN performance across k values. Optimal performance at $k = 5$ balances bias-variance tradeoff.

TABLE I
KNN ACCURACY VS. k VALUE

k	Accuracy (%)
1	89.79
3	89.47
5	90.66
7	90.34
9	89.03
11	88.71
15	88.49
20	88.17

Confusion Matrix (k=5): TP=311, TN=524, FP=38, FN=48.

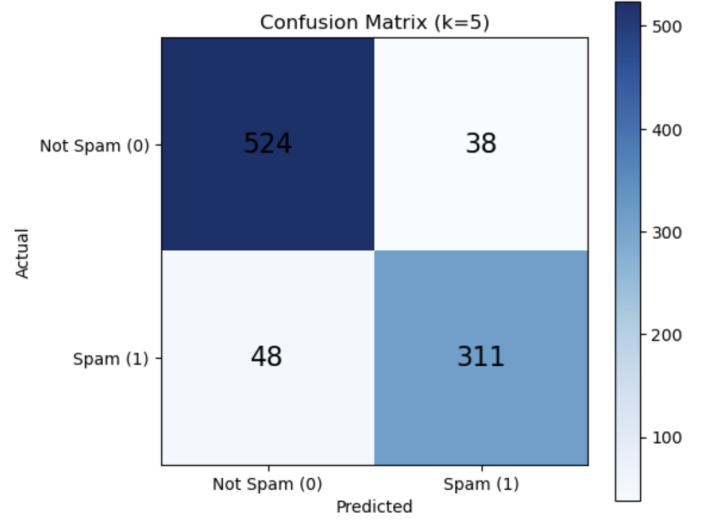


Fig. 1. Confusion matrix for K-Nearest Neighbors classification

B. Logistic Regression on Pima Indian Diabetes

We train for about 2000 Epochs and get the following results

Confusion Matrix (k=5): TP=311, TN=524, FP=38, FN=48.

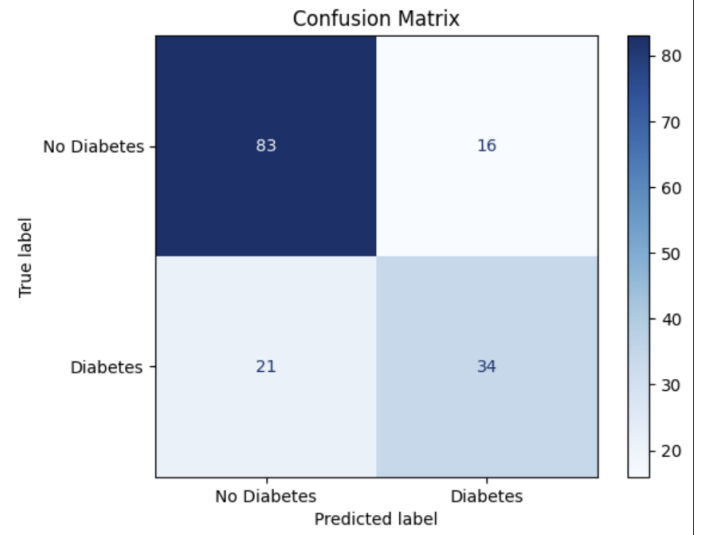


Fig. 2. Confusion matrix for Logistic regression classification

BCE LOSS Reduction: **Confusion Matrix (k=5):** TP=311, TN=524, FP=38, FN=48.

C. Artificial Neural Network on Customer Churn

Results:

- Convergence: 60-70 epochs
- Final train Accuracy: 81.06%
- Final test Accuracy: 77.22%

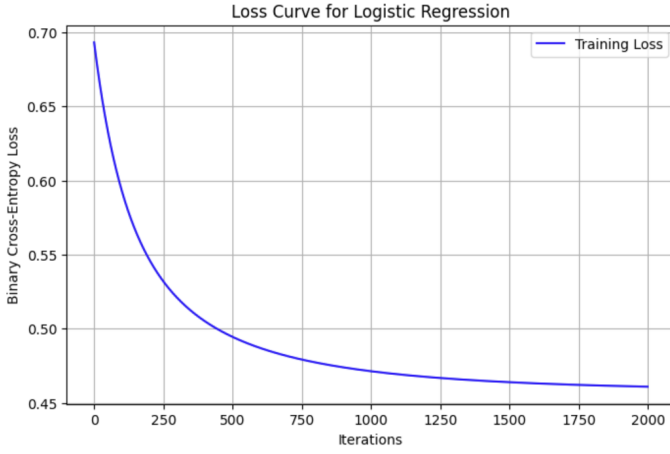


Fig. 3. Loss over epochs for logistic regression

- ROC-AUC: 0.8
- Precision: 0.5674
- Recall: 0.5963
- F1-Score: 0.5815

TABLE II
CONFUSION MATRIX FOR KNN CLASSIFICATION

Actual / Predicted	Class 0	Class 1
Class 0	865	170
Class 1	151	223

The train loss drops somewhat consistently but the test loss fluctuates a lot due to the choice of Stochastic gradient descent

D. Isolation Forest on Air Quality

The dataset was a time series dataset so we have to cyclise the time aspects as new columns.

Anomaly Statistics:

- Range of Anomaly Scores: [0.4307, 0.6822]
- Mean anomaly score: 0.4909
- For a contamination rate of 1%, the Anomaly threshold is 0.5974

Cyclical time features (sin/cos transformations) effectively captured periodic patterns.

VI. SHAP: INTERPRETABLE MODEL EXPLANATIONS

SHAP is a unified framework for interpreting machine learning predictions based on cooperative game theory. It attributes the output of a model to individual input features by computing their Shapley values. The theory was originally proposed by Lloyd Shapley in 1953 and reused to make this framework around 2017.

A. Mathematical Foundation

Given a model $f(x)$ and input features $x = [x_1, x_2, \dots, x_M]$, SHAP explains the prediction by decomposing it into additive contributions:

$$f(x) = \phi_0 + \sum_{i=1}^M \phi_i$$

where:

- ϕ_0 is the base value (expected model output),
- ϕ_i is the SHAP value for feature x_i ,
- M is the number of features.

Each SHAP value ϕ_i represents the marginal contribution of feature x_i across all possible subsets of features. Formally, the Shapley value is defined as:

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(M - |S| - 1)!}{M!} [f(S \cup \{i\}) - f(S)]$$

where:

- N is the set of all features,
- S is a subset of features excluding i ,
- $f(S)$ is the model output when only features in S are present.

This formulation ensures fairness and consistency in feature attribution, but is **computationally very expensive** due to the exponential number of subsets.

B. Kernel SHAP and Computational Challenges

In our work, we use **Kernel SHAP**, a model-agnostic approximation method suitable for custom implementations such as our own versions of KNN, logistic regression, neural networks, and isolation forests.

While it is model agnostic, Kernel SHAP is computationally intensive. For each instance, it requires multiple evaluations of the model on different feature subsets (this ends up growing exponentially with the number of features). This results in significant runtime overhead, especially when explaining large datasets or models with high dimensional inputs.

While there are alternatives such as deepshap and treeshap, we have not made use of them as our models are implemented from scratch and thus create a lot of compatibility issues.

C. Basic SHAP Visualizations

To interpret these calculated values, we have made use of the following plots:

- **Feature Importance Plot:** Ranks features by their mean absolute SHAP values across the full dataset, providing a global view of what features have a higher influence throughout the dataset.
- **Summary Plot:** Combines feature importance with feature effects. Each point represents a SHAP value for a feature in a single instance, colored by the feature's value.
- **Waterfall Plot:** Visualises how the model reached the current output starting from the expected value, and also how much of a role each feature played in it.

These plots help us interpret both global and local model behavior, despite the computational cost incurred by Kernel SHAP.

VII. SHAP EXPLAINABILITY ANALYSIS OF OUR MODELS

A. KNN Spam Detection

Top Features for Spam: Note: Below results are visualised for one instance of a SPAM email

- 1) **word_freq_recieve:** Words like recieve
- 2) **char_freq_all:** Words like all
- 3) **word_freq_remove:** Words like remove

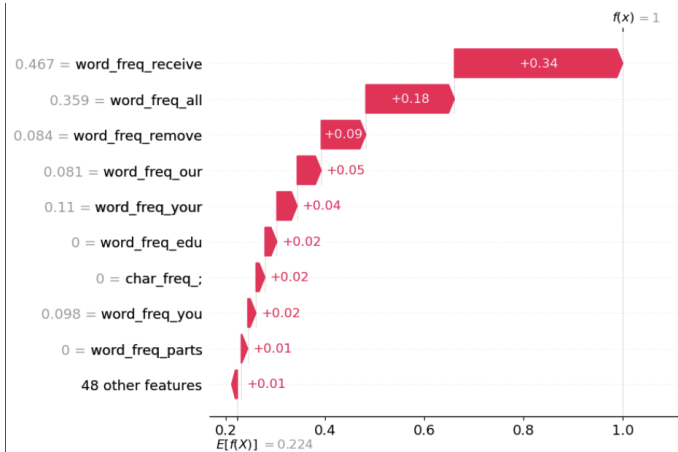


Fig. 4. How SHAP Values explain an email was predicted as SPAM

SHAP waterfall plots reveal individual predictions: legitimate emails show high values for professional terms, while spam exhibits marketing-related features.

B. Logistic Regression Diabetes Prediction

Feature importance (absolute weight magnitude):

- 1) **Glucose:** Strongest predictor
- 2) **BMI:** Obesity indicator
- 3) **Age:** Risk increases with age
- 4) **DiabetesPedigreeFunction:** Genetic factor

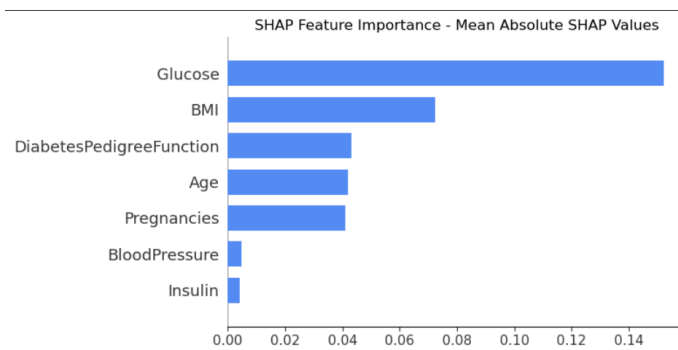


Fig. 5. Feature importance graph as per SHAP

Linear interpretability aligns with medical knowledge.

C. ANN Customer Churn

SHAP summary plots for ANN reveal non-linear interactions:

Churn Drivers:

- **Contract type:** Month-to-month contracts result in higher churn
- **Tenure:** New customers more likely to churn

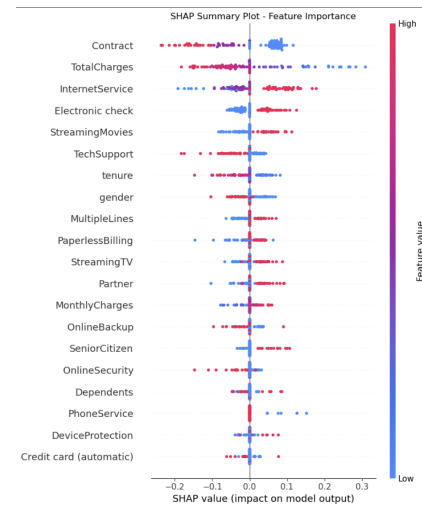


Fig. 6. A summary plot of customer churning

D. Isolation Forest Air Quality Anomalies

SHAP identified differential feature importance:

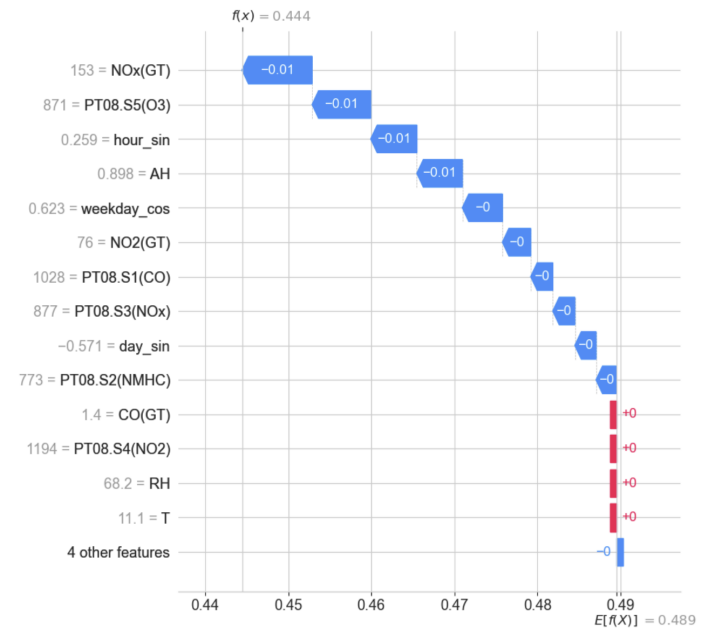


Fig. 7. SHAP values for normal instance

As shown in Figure 7, Most shap values computed for this normal row of data are 0(or very close to 0) because these features are very much within normal values and tell us that this is not an anomaly.

In Figure 8 we can clearly see that the Benzene values, and the data from the Sensor 4 which measures teh NO2 concentration is one of the top contributors to pushing this

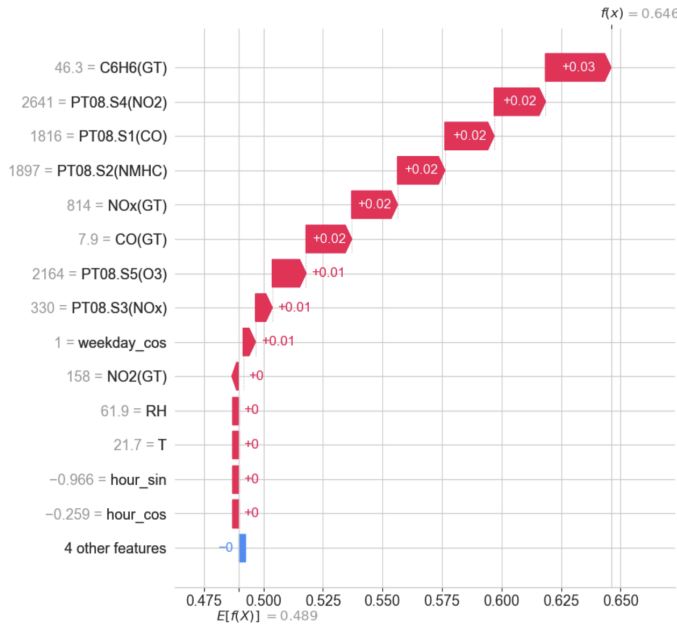


Fig. 8. SHAP values for anomaly instance

as an anomaly. Furthermore, Seeing that weekday cos feature is also contributing to this being an anomaly means that these values may be within normal ranges but not on this particular day.

VIII. CONCLUSION

This project successfully completed the process of building, evaluating, and interpreting machine learning models from scratch.

We implemented four distinct algorithms: KNN, Logistic Regression, An ANN, and Isolation Forest. The practical application of these models highlighted the critical importance of tailored data preprocessing, from handling missing values and scaling features to creating cyclical representations for time series data.

The core meaning of our project lies in bridging the gap between implementation and interpretability. By using SHAP with our own models, we were able to see their decision-making processes. Our analysis revealed clear, actionable insights: we identified key textual triggers in spam emails, confirmed known medical risk factors like Glucose and BMI for diabetes, uncovered things like Contract type and Tenure for customer churn, and differentiated the feature signatures of normal versus anomalous air quality events.

A primary challenge we faced was the high computational overhead of shap. Its model agnostic nature is computationally expensive making it unfeasible for very very large models

REFERENCES

[1] Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30.

[2] Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. In *2008 Eighth IEEE International Conference on Data Mining* (pp. 413–422). IEEE.

[3] Cramer, J. S. (2002). The origins of logistic regression. *Unpublished manuscript*.

[4] Grossi, E., & Buscema, M. (2007). Introduction to artificial neural networks. *European Journal of Gastroenterology & Hepatology*, 19(12), 1046–1054.